MAP-499
Teach. Func. Thinking
Samuel Rebelsky

Parsons Problems for Functional Programming

Salyan Karki, Julian Kim, Elene Sturua

Jacob Bell, Anna Deschamps, Eva Kapoor, Nicole Moreno González, William Pitchford, Charlotte Wade

Grinnell College

Parsons Problems are effective.

Parsons Problems (PPs) are a programming exercise where students **rearrange scrambled code snippets** to create a working program.

```
bill = 89.23
tip = bill * 0.20
total = bill + tip
numPeople = 3
perPersonCost = total / numPeople
print(perPersonCost)
```

```
def traverse_inorder(tree_node):
    if tree_node is None:
        traverse_inorder(tree_node.left)
        visit(tree_node)
```

```
SELECT * FROM test
```

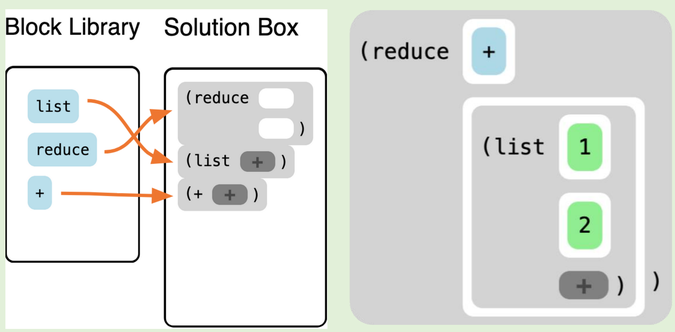
PPs reduce cognitive load by directing their focus on the logic of the code more than syntax.

However, most PP systems primarily revolve around line-based programming. What if we want students to master other kinds of sequencing?

We made PPs functional.

Nested block-based inspiration.

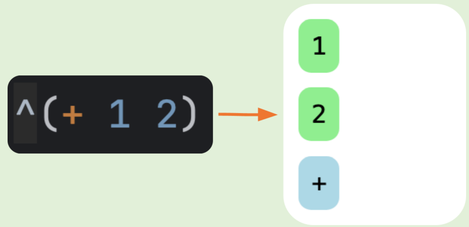
Our solution is composed of nestable blocks where students **drag and drop** sub-expressions into other expressions.



With this implementation, we can emphasize a functional approach, where functions can themselves be used as inputs to other functions.

Simple problem generation.

Parsons Problems solutions already reflect real Scheme code.
So, we implemented a method of translating **template solutions** into Parsons Problems.

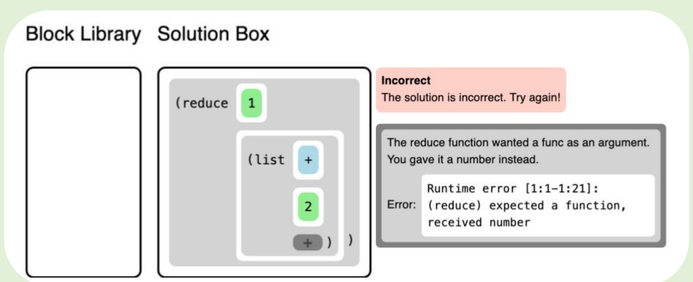


With template solutions, teachers can simply markup Scheme source code to denote which blocks should be generated.

Useful, real-time feedback.

Students shouldn't have to guess and check whether their proposed solution is right or not.

With **Scamper integration** and **error interpretation**, students receive immediate, actionable feedback upon clicking a "Check" button.



Future work.

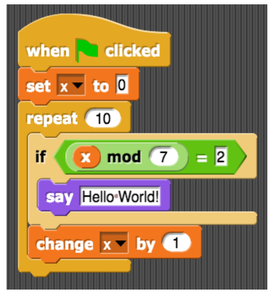
Measuring classroom effectiveness.

We expect to collaborate with CSC-151 faculty to assess the efficacy of these problems, considering both success rates on PPs and effects of the problems on student confidence, using an instrument like the Computer Science Attitudes and Beliefs Assessment [1].

Drag-and-drop Scheme programming.

Through the implementation of this prototype, we have inadvertently **created infrastructure for a drag-and-drop programming system** for Scheme, akin to Scratch or Snap [2, 3]





We plan to extend our prototype into a full drag-and-drop Scheme environment.

Future work could evaluate the pedagogical value of such a tool in an undergraduate setting.

References

[1] Andrew Hoegh and Barbara M. Moskal. 2009. Examining science and engineering students' attitudes toward computer science. In 2009 39th IEEE Frontiers in Education Conference. 1–6. https://doi.org/10.1109/FIE.2009.5350836

[2] John Maloney, Mitchel Resnick, Natalie Rusk, Brian Silverman, and Evelyn Eastmond. 2010. The Scratch Programming Language and Environment. ACM Transactions on Computing Education 10, 4 (Nov. 2010), 1–15. https://doi.org/10.1145/1868358.1868363

[3]Brian Harvey, Daniel D. Garcia, Tiffany Barnes, Nathaniel Titterton, Daniel Armendariz, Luke Segars, Eugene Lemon, Sean Morris, and Josh Paley. 2013. SNAP! (build your own blocks) (abstract only). In Proceeding of the 44th ACM technical symposium on Computer science education (SIGCSE '13). ACM, 759–759. https://doi.org/10.1145/2445196.2445507

Internal